Experiment 5
Design and Analysis of FIR Filters

Name: Corbin Ezekiel O. Gutierrez Section Code: CPE32 Date of Activity: 4/15/2026

Objectives:

After completing this experiment, students should be able to:

1. Understand the concept of **Finite Impulse Response (FIR) filters**
2. Design **low-pass, high-pass, and band-pass FIR filters**
3. Analyze:
    - Impulse response
    - Frequency response
4. Apply FIR filters to signals using Python
5. Visualize filtering effects in time and frequency domains

Background Theory

Finite Impulse Response (FIR) filters are digital filters with a finite-duration impulse response. They are inherently stable and can be designed to have exact linear phase, which is important in signal processing applications.

General FIR equation:

$$y[n] = \Sigma\, h[k]\, x[n - k],$$

where $k = 0$ to $M - 1$

Materials / Requirements

- PC or laptop

Software

- Python 3.x
- NumPy
- SciPy
- Matplotlib

Experimental Parameters

- Sampling frequency: 1000 Hz
- Filter order: 51
- Test signal: Sum of sinusoids (low, mid, high frequencies)

Experiment Tasks

Part A: Generate Test Signal

Generate a composite test signal consisting of low, mid, and high frequency components.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, lfilter, freqz

# Sampling parameters
fs = 1000  # Hz
t = np.arange(0, 0.1, 1/fs)

# Test signal with three frequencies
signal = (np.sin(2*np.pi*50*t) +
        np.sin(2*np.pi*150*t) +
        np.sin(2*np.pi*300*t))

plt.figure(figsize=(10,4))
plt.plot(t, signal)
plt.title("Original Signal (Time Domain)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```

Part B: FIR Low-Pass Filter

Design a low-pass FIR filter and analyze its frequency response.

Specifications

- Cutoff frequency = 100 Hz

```
# Low-pass filter
lpf = firwin(numtaps=51, cutoff=100, fs=fs)

filtered_lpf = lfilter(lpf, 0.1, signal)

# Frequency response
w, h = freqz(lpf, worN=8000, fs=fs)

plt.figure(figsize=(10,4))
plt.plot(w, abs(h))
plt.title("Low-Pass FIR Filter Frequency Response")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid()
plt.show()
```

Part C: FIR High-Pass Filter

Design a high-pass FIR filter and analyze its frequency response.

Specifications

- Cutoff frequency = 200 Hz

```
# High-pass filter
hpf = firwin(numtaps=51, cutoff=200, fs=fs, pass_zero=False)

filtered_hpf = lfilter(hpf, 0.1, signal)

w, h = freqz(hpf, worN=8000, fs=fs)

plt.figure(figsize=(10,4))
plt.plot(w, abs(h))
plt.title("High-Pass FIR Filter Frequency Response")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid()
plt.show()
```

Part D: FIR Band-Pass Filter

Design a band-pass FIR filter and analyze its frequency response.

Specifications

- Passband: 100 Hz – 250 Hz

```
# Band-pass filter
bpf = firwin(numtaps=51, cutoff=[100, 250], fs=fs, pass_zero=False)

filtered_bpf = lfilter(bpf, 0.1, signal)

w, h = freqz(bpf, worN=8000, fs=fs)

plt.figure(figsize=(10,4))
plt.plot(w, abs(h))
plt.title("Band-Pass FIR Filter Frequency Response")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid()
plt.show()
```

Part E: Time-Domain Representation

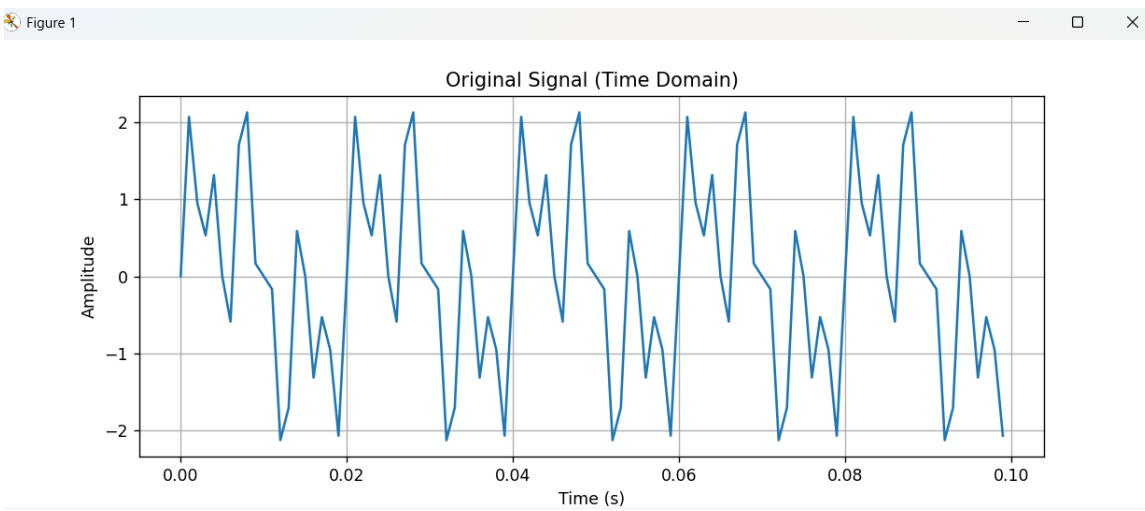
Plot the original signal and the filtered signals.

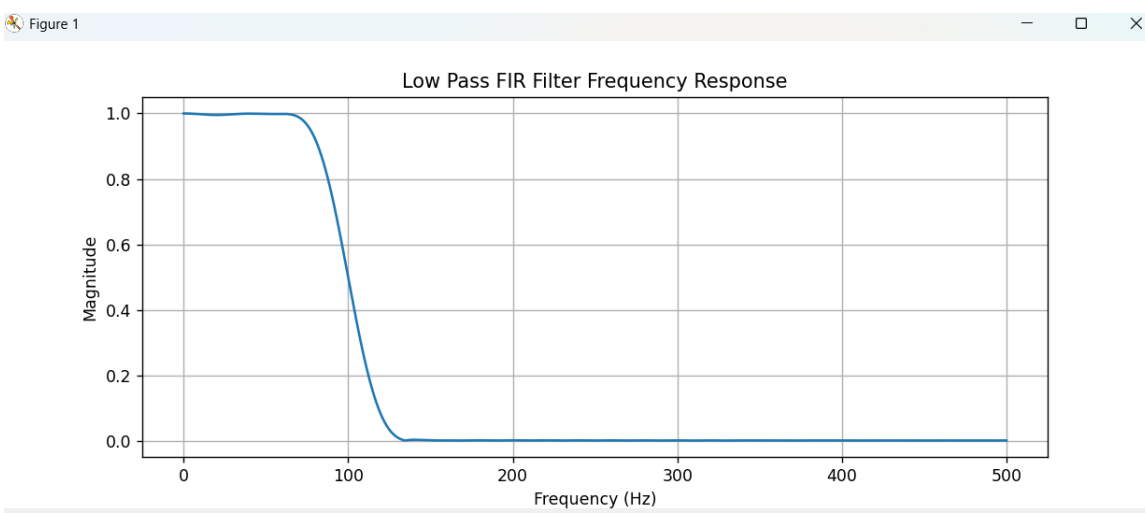
```
plt.figure(figsize=(12,6))
plt.subplot(4,1,1)
plt.plot(signal)
plt.title("Original Signal")

plt.subplot(4,1,2)
plt.plot(filtered_lpf)
plt.title("Low-Pass Filtered Signal")

plt.subplot(4,1,3)
plt.plot(filtered_hpf)
plt.title("High-Pass Filtered Signal")

plt.subplot(4,1,4)
plt.plot(filtered_bpf)
plt.title("Band-Pass Filtered Signal")
```

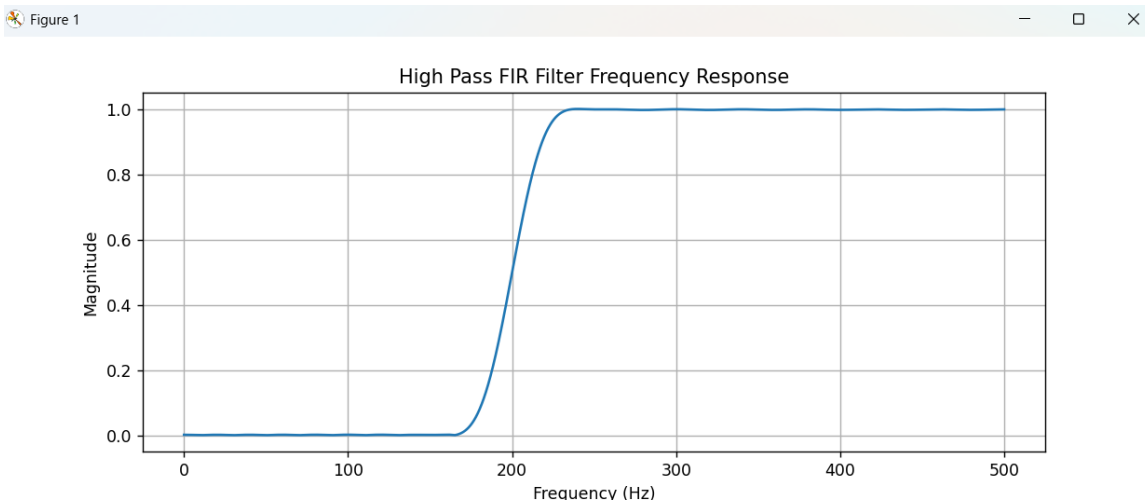
Post Laboratory Exercises

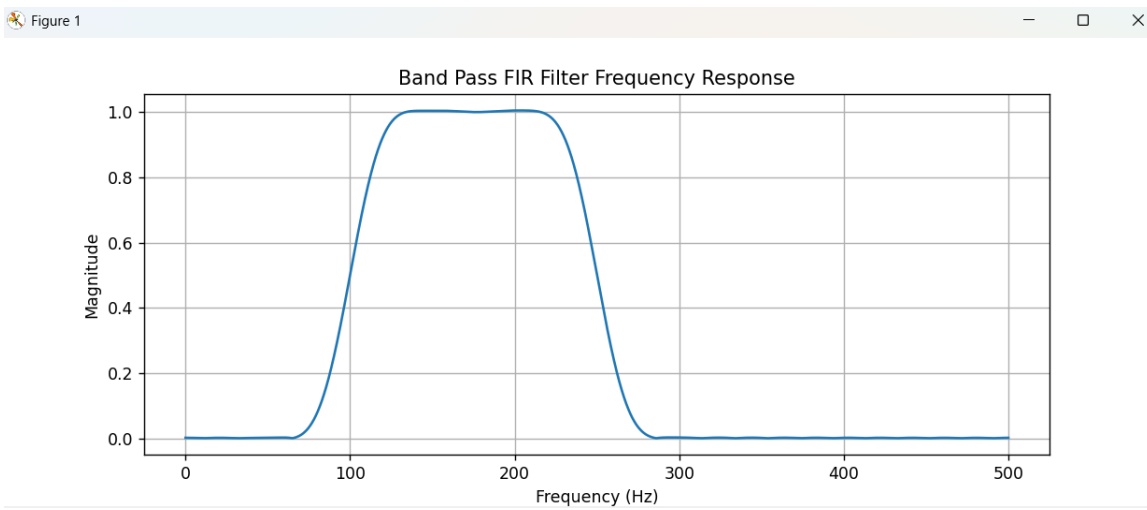
1. Increase the filter order to 101 and compare the frequency responses.
2. Change the cutoff frequencies and observe the effect on the output signal.
3. Modify the sampling frequency and analyze filter performance changes.
4. Demonstrate the effect of aliasing when cutoff frequency exceeds Nyquist frequency.
5. Apply the FIR filters to a real-world signal (speech or ECG).

Results:









Observation/Discussion/Conclusion:

In this experiment, Finite Impulse Response (FIR) filters were successfully designed and implemented using Python. A composite signal was filtered using low-pass, high-pass, and band-pass configurations. Overall, the experiment demonstrates the importance of FIR filters in digital signal processing, especially in applications requiring stability and phase preservation.